

Multimodal Visual Understanding + SLAM Navigation

Multimodal Visual Understanding + SLAM Navigation

1. Course Content
2. Preparation
 - 2.1 Content Description
 - 2.2 Starting the Agent
 - 2.3 Configuring the Map Mapping File
3. Running Example
 - 3.1 Starting the Program
 - 3.2 Test Cases
4. Source Code Analysis

1. Course Content

- Basic: Run example programs, combining the robot's visual understanding capabilities with SLAM navigation for integrated tasks.
- Advanced: Master the key source code introduced in this section.

2. Preparation

2.1 Content Description

This section of the course uses the Jetson Orin NX as an example. For Raspberry Pi and Jetson Nano boards, you need to open a terminal on the host machine, then enter the command to enter the Docker container. After entering the Docker container, enter the commands mentioned in this section of the course in the terminal. Instructions on how to access the Docker container from the host machine can be found in the product tutorial [0. Instructions and Installation Steps], specifically the section [Accessing the Robot's Docker (For Jetson Nano and Raspberry Pi 5 users)]. For Orin and NX boards, simply open a terminal and enter the commands mentioned in this section.

2.2 Starting the Agent

Note: If the agent is already running, there is no need to start it again.

Enter the following command in the vehicle terminal:

```
sh start_agent.sh
```

The terminal will print the following information, indicating a successful connection:

```
Jetson@yahboom: ~  
Jetson@yahboom: ~ - 177x26  
Jetson@yahboom:~$ ./open_agent.sh  
[1749538760.063253] Info | TermiosAgentLinux.cpp | init | running... | fd: 3  
[1749538760.064012] Info | Root.cpp | set_verbose_level | logger setup | verbose_level: 4  
[1749538760.941420] Info | Root.cpp | create_client | create | client_key: 0x0DA64EFC, session_id: 0x81  
[1749538760.941537] Info | SessionManager.hpp | establish_session | session established | client_key: 0x0DA64EFC, address: 0  
[1749538760.979971] Info | ProxyClient.cpp | create_participant | participant created | client_key: 0x0DA64EFC, participant_id: 0x000(1)  
[1749538760.983835] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x000(2), participant_id: 0x000(1)  
[1749538760.988244] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x000(3), participant_id: 0x000(1)  
[1749538760.993117] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x000(5), publisher_id: 0x000(3)  
[1749538760.997675] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x001(2), participant_id: 0x000(1)  
[1749538761.001121] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x001(3), participant_id: 0x000(1)  
[1749538761.007277] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x001(5), publisher_id: 0x001(3)  
[1749538761.010634] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x002(2), participant_id: 0x000(1)  
[1749538761.014296] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x002(3), participant_id: 0x000(1)  
[1749538761.020171] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x002(5), publisher_id: 0x002(3)  
[1749538761.023939] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x003(2), participant_id: 0x000(1)  
[1749538761.029173] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x003(3), participant_id: 0x000(1)  
[1749538761.034377] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x003(5), publisher_id: 0x003(3)  
[1749538761.038946] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x004(2), participant_id: 0x000(1)  
[1749538761.042215] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x004(3), participant_id: 0x000(1)  
[1749538761.048422] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x004(5), publisher_id: 0x004(3)  
[1749538761.051660] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x005(2), participant_id: 0x000(1)  
[1749538761.057494] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x005(3), participant_id: 0x000(1)  
[1749538761.062183] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x005(5), publisher_id: 0x005(3)  
[1749538761.066842] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x006(2), participant_id: 0x000(1)  
[1749538761.071217] Info | ProxyClient.cpp | create_subscriber | subscriber created | client_key: 0x0DA64EFC, subscriber_id: 0x000(4), participant_id: 0x000(1)
```

[!NOTE]

Note: To experience this section, you need to have built at least one grid map according to the LiDAR section of the course.

2.3 Configuring the Map Mapping File

- Connect to the robot's desktop via VNC and start the navigation node using the following commands:

```
ros2 launch M3Pro_navigation base_bringup.launch.py
```

```
ros2 launch M3Pro_navigation navigation2.launch.py
```

- Start rviz on the robot:

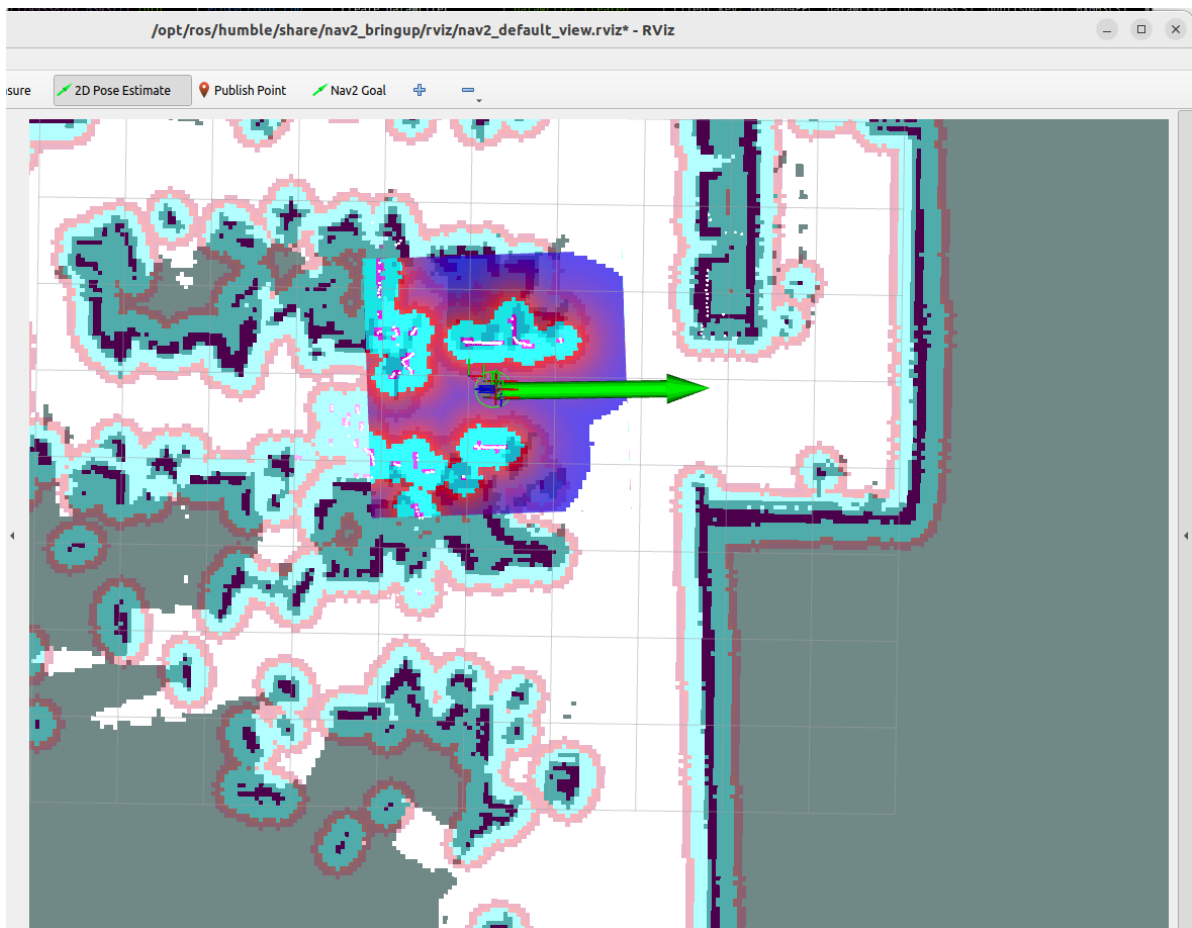
```
ros2 launch M3Pro_navigation nav_rviz.launch.py
```

- Alternatively, you can start the display on the virtual machine; there is no need to start the display window repeatedly.

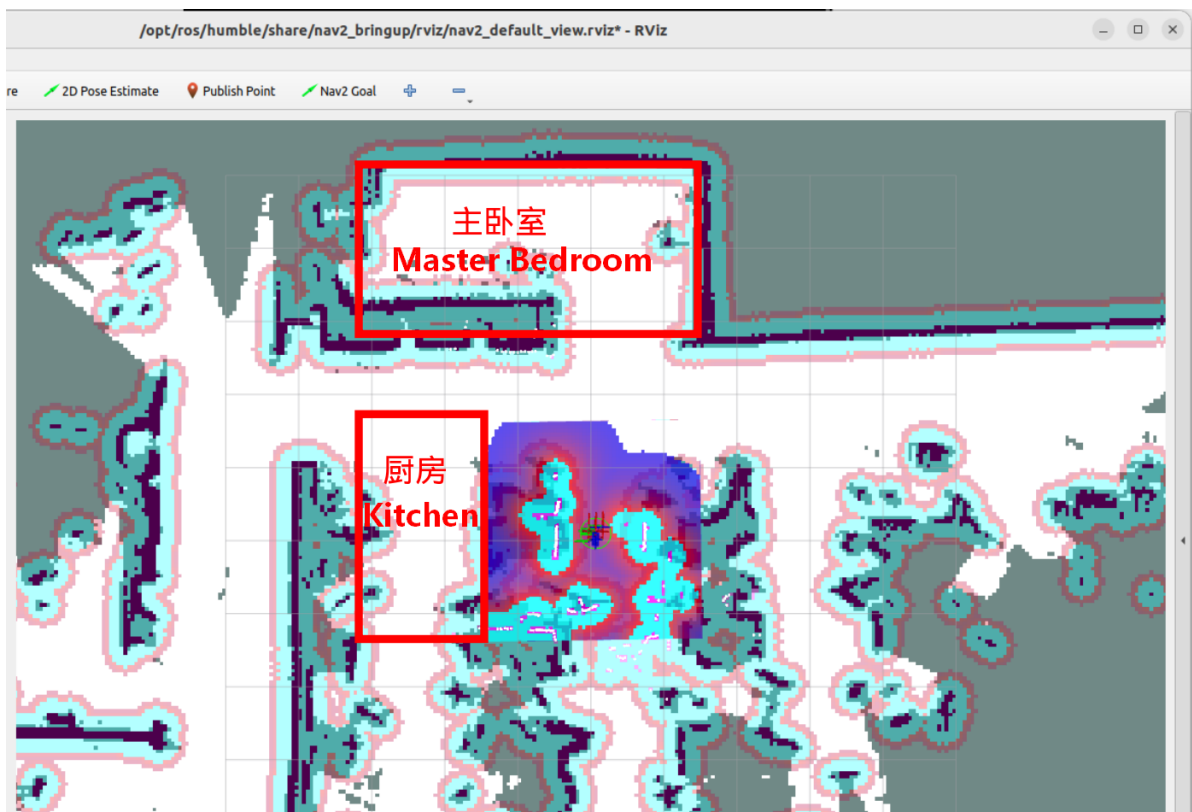
```
ros2 launch slam_view nav_rviz.launch.py
```

Afterward, the rviz2 visualization interface will open. Click **2D Pose Estimate** in the toolbar above to enter the selection state, and roughly mark the robot's position and orientation on the map.

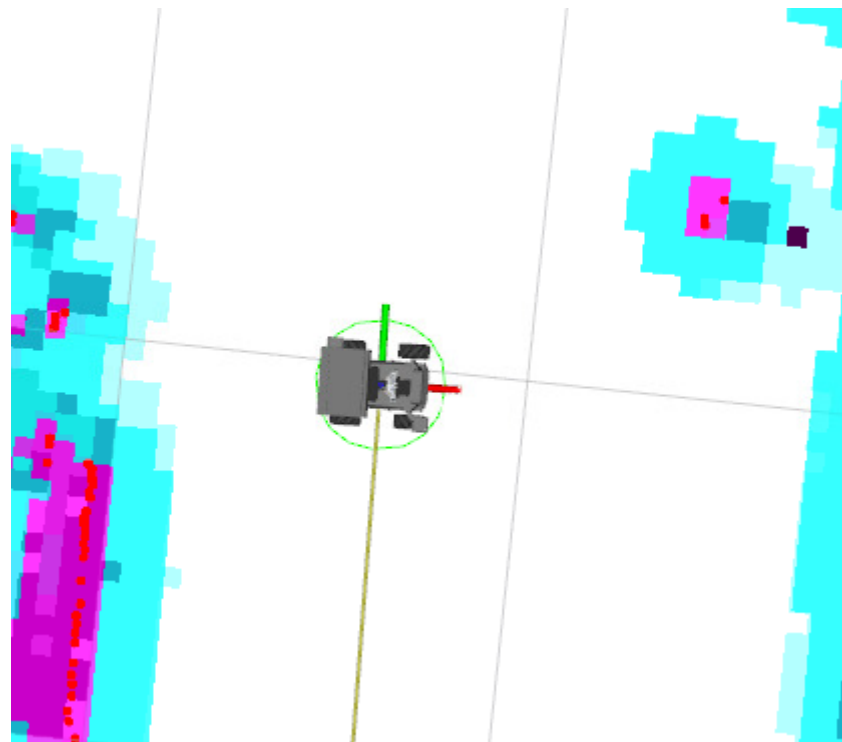
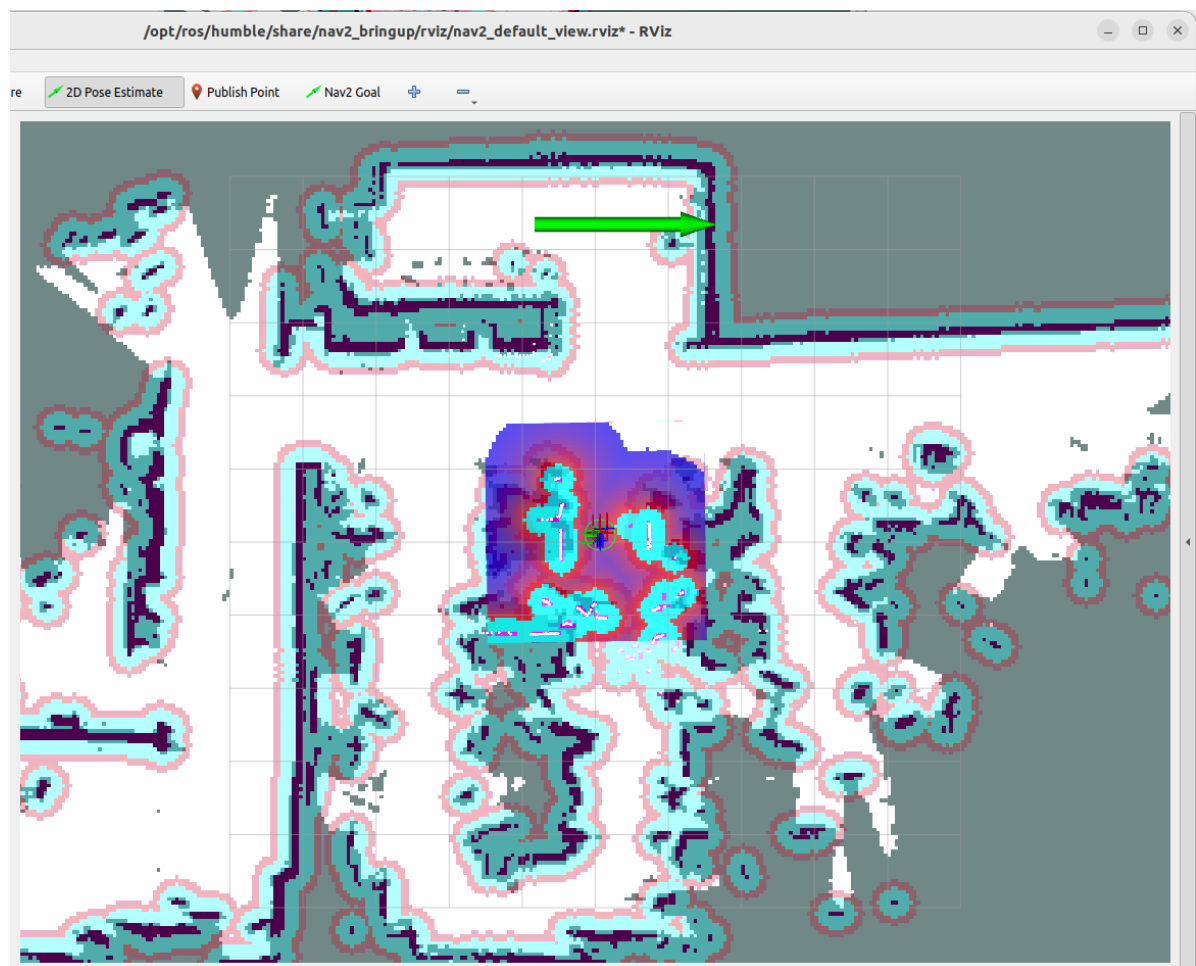
The robot model will be displayed in the map, as shown below:



We can name any precise point on the map. Here, we use "Master Bedroom" and "Kitchen" as examples.



As shown in the figure below, we first click the **Nav2 Goal** tool to navigate the robot to the target point we need to mark.



- Run the following command in the terminal to obtain the current robot's pose information in the map coordinate system:

```
ros2 run tf2_ros tf2_echo map base_footprint
```

```

jetson@yahboom:~$ ros2 run tf2_ros tf2_echo map base_footprint
[INFO] [1754377006.239762954] [tf2_echo]: Waiting for transform map -> base_footprint: Invalid frame ID "map" passed to canTransform argument target_frame - frame does not exist
[INFO] [1754377008.204300151] [tf2_echo]: Waiting for transform map -> base_footprint: Invalid frame ID "map" passed to canTransform argument target_frame - frame does not exist
At time 1754377008.700000000
- Translation: [3.974, -2.634, 0.000]
- Rotation: in Quaternion [0.000, 0.000, -0.688, 0.726]
- Rotation: in RPY (radian) [0.000, 0.000, -1.516]
- Rotation: in RPY (degree) [0.000, 0.000, -86.867]
- Matrix:
  0.055  0.999  0.000  3.974
 -0.999  0.055 -0.000 -2.634
 -0.000  0.000  1.000  0.000
  0.000  0.000  0.000  1.000
At time 1754377009.734000000
- Translation: [3.974, -2.634, 0.000]
- Rotation: in Quaternion [0.000, 0.000, -0.688, 0.726]
- Rotation: in RPY (radian) [0.000, 0.000, -1.517]
- Rotation: in RPY (degree) [0.000, 0.000, -86.943]
- Matrix:
  0.053  0.999  0.000  3.974
 -0.999  0.053 -0.000 -2.634
 -0.000  0.000  1.000  0.000

```

Open the `map_mapping.yaml` map mapping file (you can open it using VNC, VS Code, command line, or any other method):

- Here's an example of opening the file via the command line:

```
nano ~/M3Pro_ws/multi_brains_file/map_mapping.yaml
```

```

GNU nano 6.2 /home/jetson/M3Pro_ws/multi_brains_file/map_mapping.yaml
#根据实际的场景环境，自定义地图中的区域，可以添加任意个区域，注意和大模型的地图映射保持一致即可
#According to the actual scene environment, customize the areas in the map. You can add any number of areas, just make sure
#地图映射Map mapping

common_map_areas: #常规导航 common navigation
A:
  name: 'xxx'
  position:
    x: 1.824
    y: -0.838
    z: 0.0
  orientation:
    x: 0.0
    y: 0.0
    z: -0.663
    w: 0.748
B:
  name: 'xxx'
  position:
    x: 1.488
    y: 0.661
    z: 0.0
  orientation:
    x: 0.0
    y: 0.0
    z: 0.725
    w: 0.688

```

- Modify the symbolic pose under the `common_map_areas` field. `name` is the location name. Fill in the previously obtained pose information into the `position` and `orientation` fields.

```

#根据实际的场景环境，自定义地图中的区域，可以添加任意个区域，注意和大模型的地图映射保持一致即可
#According to the actual scene environment, customize the areas in the map. You can add any number of areas, just make sure they are consistent with the map mapping of the large model
#地图映射Map mapping

```

```

common_map_areas: #常规导航 common navigation
A:
  name: 'Master Bedroom'
  position:
    x: 3.974
    y: -2.634
  orientation:
    x: 0.0
    y: 0.0
    z: -0.688

```

```

w: 0.726
B:
  name: 'xxx'
  position:
    x: 1.488
    y: 0.661
    z: 0.0
  orientation:
    x: 0.0
    y: 0.0
    z: 0.725
  w: 0.688

```

After the modifications are complete, saving the file will take effect immediately. 2.4 Configuring Map Mapping Variables in Dify

- After configuring the map mapping file as described above, we need to let the AI large language model know the relationship between the locations and symbols in these maps.
- Start the Dify service (if already started, no need to restart):

```
bringup_dify
```

```

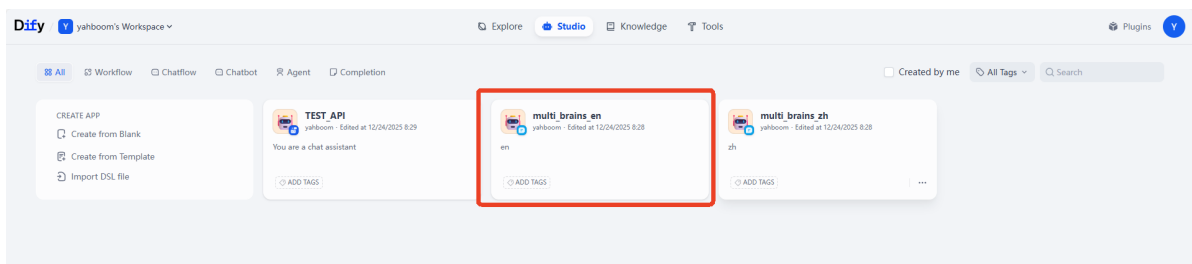
pi@raspberrypi: ~
pi@raspberrypi: ~ 80x24
pi@raspberrypi:~ $ bringup_dify
[+] up 14/14
✓ Network docker_default Created 0.0s
✓ Network docker_ssrf_proxy_network Created 0.1s
✓ Container docker_init_permissions-1 Exited 7.8s
✓ Container docker_sandbox-1 Created 2.9s
✓ Container docker_web-1 Created 2.9s
✓ Container docker_redis-1 Created 2.9s
✓ Container docker_weaviate-1 Created 2.8s
✓ Container docker_ssrf_proxy-1 Created 2.9s
✓ Container docker_db_postgres-1 Healthy 7.8s
✓ Container docker_worker-1 Created 0.3s
✓ Container docker_api-1 Created 0.3s
✓ Container docker_worker_beat-1 Created 0.3s
✓ Container docker_plugin_daemon-1 Created 0.2s
✓ Container docker_nginx-1 Created 0.3s
pi@raspberrypi:~ $

```

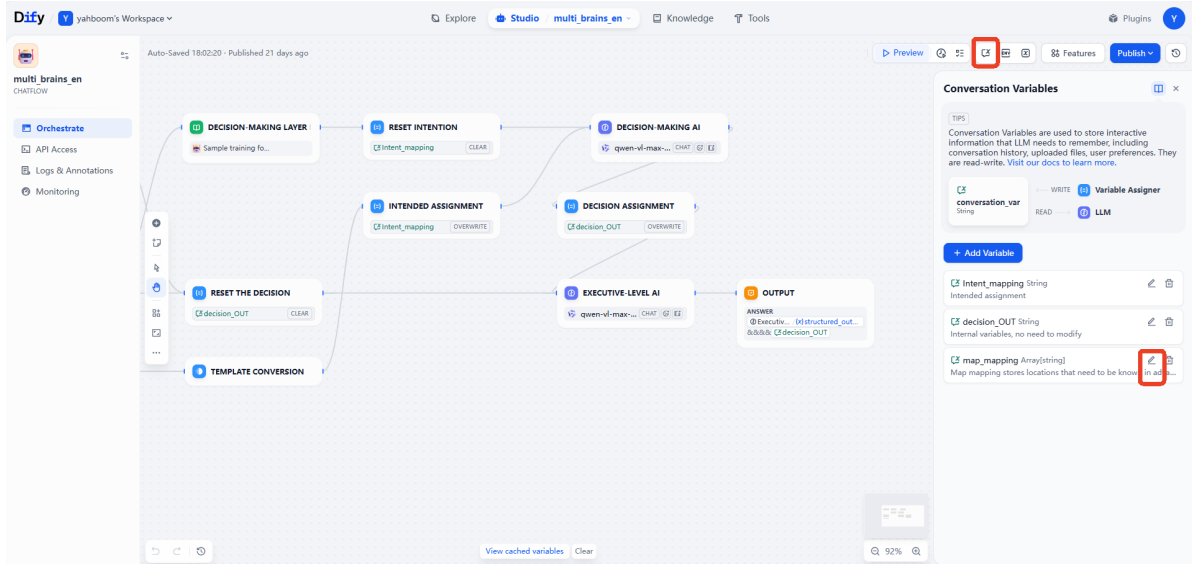
- Enter the vehicle's IP address directly in the browser's address bar to access the Dify management page, then click to select the corresponding AI application.

[!NOTE]

- International users: multi_brains_en



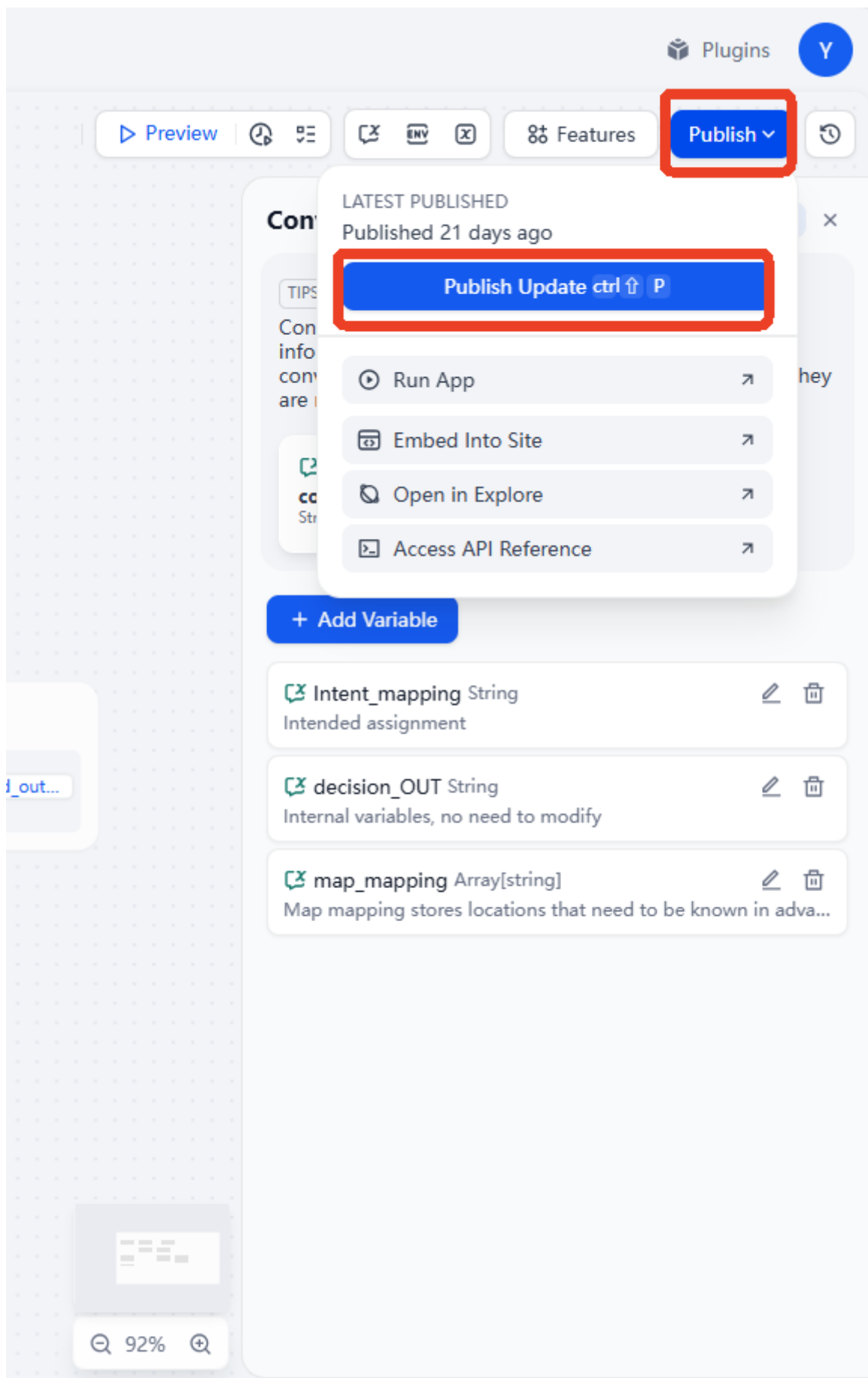
- Click to select `Session variables` in the upper right corner, then click the edit button for the `map_mapping` variable.



- In the pop-up **Edit Session variables** window, edit the mapping relationship according to the settings in the previous map mapping file, and then click Save.

The screenshot shows the 'Edit Conversation Variable' pop-up window. The 'Name' field is 'map_mapping' and the 'Type' is 'array[string]'. The 'Default Value' section is highlighted with a red box and contains two entries: 'School: A' and 'Library: B'. Below this is a '+ Add Value' button. The 'Description' field contains the text: 'Map mapping stores locations that need to be known in advance by AI'. At the bottom right, the 'Save' button is highlighted with a red box.

- Finally, remember to click Publish -> Publish Update to save the changes.



3. Running Example

3.1 Starting the Program

On the vehicle's terminal, enter the command to start the AI intelligent agent system:

```
ros2 launch multi_brains llm_agent_control.launch.py`
```

Or you can use the shortcut command:

```
multi_brains
```

- Start the navigation command on the vehicle's onboard computer:

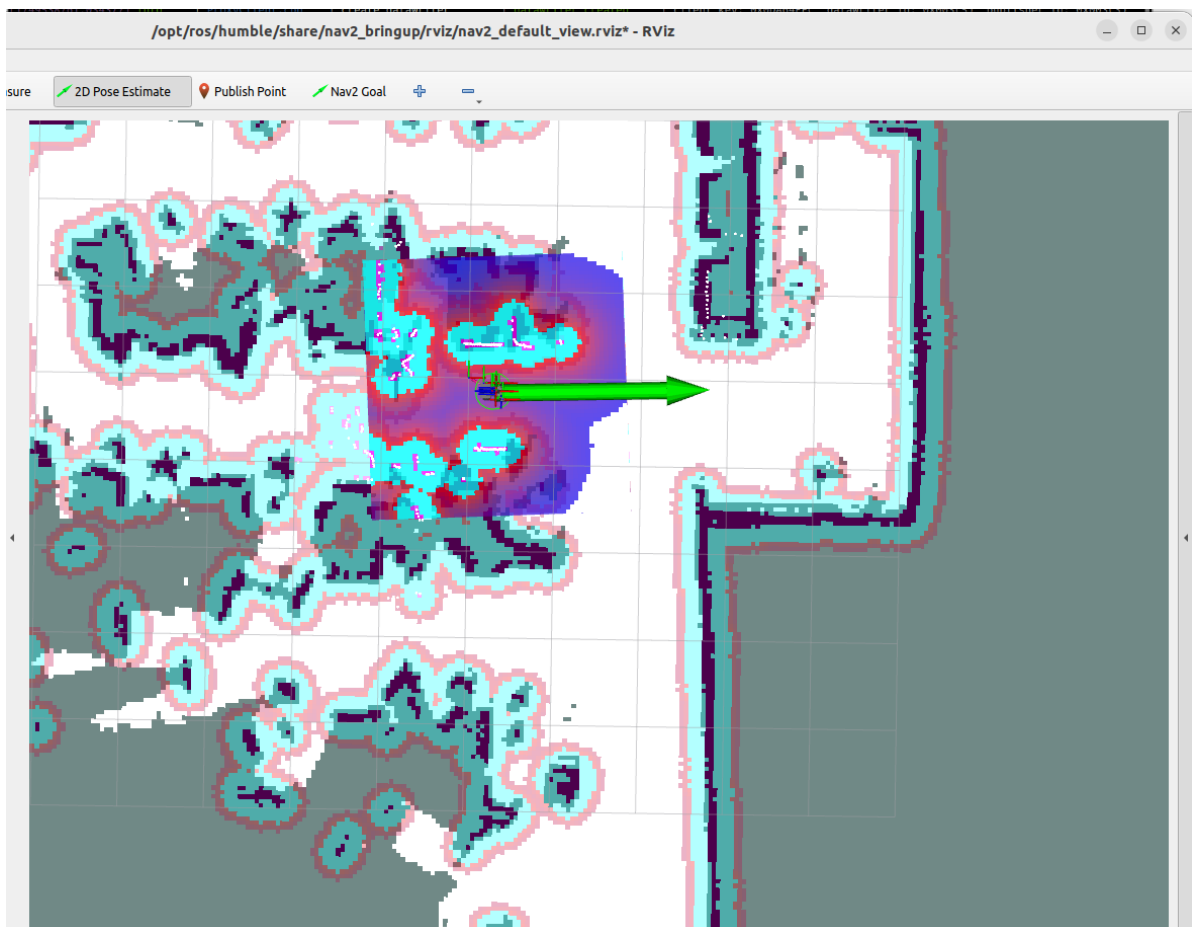
```
ros2 launch M3Pro_navigation base_bringup.launch.py
```

```
ros2 launch M3Pro_navigation navigation2.launch.py
```

- Start rviz on the robot:

```
ros2 launch M3Pro_navigation nav_rviz.launch.py
```

- Then, follow the procedure for starting the navigation function to initialize the positioning. This will open the rviz2 visualization interface. Click on **2D Pose Estimate** in the toolbar above to enter the selection state. Mark the approximate position and orientation of the robot on the map. After initializing the positioning, the preparation work is complete.



3.2 Test Cases

The cases are for reference only; provide instructions according to your needs. - Please remember your current location first, then navigate to the kitchen and the master bedroom in sequence, remembering the items you see. Finally, return to your starting position and tell me what you saw in those two places?

- Wake up the robot and issue commands. The execution layer large model will execute subtasks according to the task steps planned by the decision layer model: 1. Navigate to the "master bedroom," then observe the items in the environment. Place a pen in the simulated "master bedroom," and place a pack of toilet paper in the simulated "kitchen":

```
jetson@yahboom: ~  
[asr-5] [INFO] [1750385414.028556399] [asr]: 0  
[asr-5] [INFO] [1750385414.119921759] [asr]: 0  
[asr-5] [INFO] [1750385414.210165105] [asr]: 0  
[asr-5] [INFO] [1750385414.300471621] [asr]: 0  
[asr-5] [INFO] [1750385414.390265068] [asr]: 0  
[asr-5] [INFO] [1750385414.481523321] [asr]: 0  
[asr-5] [INFO] [1750385414.571777356] [asr]: 0  
[asr-5] [INFO] [1750385414.662565773] [asr]: 0  
[asr-5] [INFO] [1750385414.754897462] [asr]: 0  
{'load_data': '0.028', 'extract_feat': '0.036', 'forward': '4.403', 'batch_size': '1', 'rtf': '0.263'},  
rtf_avg: 0.263: 100% | 1/1 [00:04<00:00, 4.40s/it]  
rtf_avg: 0.263: 100% | 1/1 [00:04<00:00, 4.40s/it]  
[asr-5] [INFO] [1750385419.211170831] [asr]: 请你先记住现在的位置然后依次导航去厨房和主卧室记住你所看到的  
物品最后返回出发时的位置告诉我你都在那两个地方看到了什么  
[asr-5] [INFO] [1750385419.211922083] [asr]: okay 😊, let me think for a moment...  
[model_service-3] [INFO] [1750385423.231742756] [model_service]: The upcoming task to be carried out: 1.  
调用记录当前位置函数记录位置2.导航去厨房, 调用获取当前视角图像函数获取图像并记录看到的物品3.导航去主卧室  
, 调用获取当前视角图像函数获取图像并记录看到的物品4.调用返回初始位置函数回到出发时的位置5.总结在厨房和主  
卧室看到的物品并告知用户  
[model_service-3] [INFO] [1750385427.315891765] [model_service]: "action": ['get_current_pose()'], "resp  
onse": 好的, 我现在就记住现在的位置, 就像记住了一个秘密基地一样。接下来, 我要去厨房和主卧室探险啦!  
[action_service-4] [INFO] [1750385436.995532898] [action_service]: Recorded Pose - Position: x=-0.070540  
17845186356, y=0.07985647398462059, z=0.0  
[action_service-4] [INFO] [1750385436.996906727] [action_service]: Recorded Pose - Orientation: x=0.0, y  
=0.0, z=-0.3472589691926351, w=0.9377692724307342  
[model_service-3] [INFO] [1750385441.388327622] [model_service]: "action": ['navigation(G)'], "response"  
: 我已经记住了当前位置, 现在准备导航去厨房。就像一个勇敢的探险家, 出发!
```

- The robot executes the steps according to the output process of the decision layer as follows:

```
[model_service-3] [INFO] [1750385441.388327622] [model_service]: "action": ['navigation(G)'], "response"  
: 我已经记住了当前位置, 现在准备导航去厨房。就像一个勇敢的探险家, 出发!  
[action_service-4] [INFO] [1750385449.596979417] [action_service]: navigation Goal accepted  
[action_service-4] [INFO] [1750385516.629335533] [action_service]: Navigation succeeded!  
[model_service-3] [INFO] [1750385520.996419381] [model_service]: "action": ['seewhat()'], "response": 我  
已经到达厨房了, 现在开始用我的火眼金睛观察这里有什么物品。  
[model_service-3] [INFO] [1750385537.403688769] [model_service]: "action": [], "response": 在厨房里, 我  
看到了一支笔静静地躺在地上。看来这里除了烹饪工具, 还藏着一些小秘密呢! 接下来, 我要去主卧室看看了。  
[model_service-3] [INFO] [1750385555.608727072] [model_service]: "action": ['navigation(A)'], "response"  
: 我已经记录了在厨房看到的物品, 现在准备前往主卧室。就像一个好奇的侦探, 继续我的探索之旅!  
[action_service-4] [INFO] [1750385565.962718690] [action_service]: navigation Goal accepted  
[action_service-4] [INFO] [1750385595.045950277] [action_service]: Navigation succeeded!  
[model_service-3] [INFO] [1750385599.609402332] [model_service]: "action": ['seewhat()'], "response": 我  
已经到达主卧室了, 现在开始仔细观察这里有什么物品。  
[model_service-3] [INFO] [1750385611.106135108] [model_service]: "action": [], "response": 在主卧室里,  
我看到了一盒橙色包装的纸巾放在光滑的地面上。这个地方看起来很整洁, 纸巾显得格外显眼。现在, 我要返回出发时  
的位置了。  
[model_service-3] [INFO] [1750385632.170785773] [model_service]: "action": ['navigation(zero)'], "respon  
se": 我已经记录了在主卧室看到的物品, 现在准备返回出发时的位置。就像完成了一次小旅行, 满载而归!  
[action_service-4] [INFO] [1750385642.582316949] [action_service]: navigation Goal accepted  
[action_service-4] [INFO] [1750385672.035012715] [action_service]: Navigation succeeded!  
[model_service-3] [INFO] [1750385679.513325513] [model_service]: "action": [], "response": 我已经回到了  
出发时的位置。让我来总结一下我在厨房和主卧室看到的物品吧! 在厨房, 我看到了一支笔静静地躺在地上; 而在主卧  
室, 我则发现了一盒橙色包装的纸巾放在光滑的地面上。这两个地方各有特色, 是不是很有趣呢?  
[model_service-3] [INFO] [1750385706.296938429] [model_service]: "action": ['finishtask()'], "response":  
我已经完成了全部任务, 告诉了你在厨房和主卧室看到的物品。如果还有其他需要帮忙的地方, 随时告诉我哦!
```

4. Source Code Analysis

- Robot action source code path:

```
~/M3Pro_ws/src/multi_brains/multi_brains/action_service.py
```

action_service.py program:

Case 1 uses the `seewhat`, `navigation`, `load_target_points`, and `get_current_pose` methods in the `CustomActionServer` class. `seewhat` has been explained in the **Multimodal Visual Understanding + Robotic Arm Grasping** section. This section explains the newly introduced `navigation`, `load_target_points`, and `get_current_pose` functions.

`init_ros_communication` initialization function:

- Creates a nav2 navigation client to request the ros2 navigation action server for subsequent sending of navigation target point requests; creates a TF listener to listen to the coordinate transformation between map and base_footprint.

```
# Create a navigation action client to request the navigation action server
self.navclient = ActionClient(self, NavigateToPose, 'navigate_to_pose')
# Create a TF listener to listen for coordinate transformations
self.tf_buffer = Buffer()
self.tf_listener = TransformListener(self.tf_buffer, self)
```

`load_target_points` function:

- This function is responsible for loading the target point coordinates from the `map_mapping.yaml` map mapping file and creating a navigation dictionary to store characters and their corresponding map coordinates. Each point coordinate is of type `PoseStamped`.

```
def load_target_points(self):
    """
    加载地图映射文件 /Load map mapping file
    """
    self.navpose_dict = {}
    self.road_net_dict = {}
    with open(self.map_mapping_file, "r") as file:
        full_target_points = yaml.safe_load(file)
        common_target_points = full_target_points.get("common_map_areas",
    {})

        road_net_target_points =
    full_target_points.get("road_net_map_areas", {})

        for name, data in common_target_points.items():#加载常规导航点 / Load
    common navigation points
            pose = PoseStamped()
            pose.header.frame_id = "map"
            pose.pose.position.x = data["position"]["x"]
            pose.pose.position.y = data["position"]["y"]
            pose.pose.position.z = data["position"]["z"]
            pose.pose.orientation.x = data["orientation"]["x"]
            pose.pose.orientation.y = data["orientation"]["y"]
            pose.pose.orientation.z = data["orientation"]["z"]
            pose.pose.orientation.w = data["orientation"]["w"]
            self.navpose_dict[name] = pose
```

`_normal_navigation` function:

- Receives a character parameter (corresponding to the characters in the map mapping described above), parses the coordinates corresponding to that character from the dictionary, and uses the `self.navclient` navigation client object to request the ROS2 navigation action server. When the navigation action server returns a value of 4, it indicates successful navigation; other values indicate failure (possibly due to obstacles, planning failures, etc.). After the navigation is complete, the function provides feedback on the action execution result to the large language model.

```
def __normal_navigation(self, point_name)->None:
    '''常规导航功能 / Normal navigation function '''
    self.navigation_finish_flag = False
    self.goal_handle = None
    self.result = None
    self.res = None
    point_name = point_name.strip("\'\"")
    if point_name not in self.navpose_dict:
        self.get_logger().error(f"Target point '{point_name}' does not exist
in the navigation dictionary." )
        return None

    if self.first_record:
        # 出发前记录当前在全局地图中的坐标(只有在每个任务周期的第一次执行时才会记录)/
before starting a new task, record the current pose in the global map
        transform = self.tf_buffer.lookup_transform(
            "map", "base_footprint", rclpy.time.Time()
        )
        pose = PoseStamped()
        pose.header.frame_id = "map"
        pose.pose.position.x = transform.transform.translation.x
        pose.pose.position.y = transform.transform.translation.y
        pose.pose.position.z = 0.0
        pose.pose.orientation = transform.transform.rotation
        self.navpose_dict["zero"] = pose
        self.road_net_dict["zero"] = pose
        self.first_record = False

    # 获取目标点坐标 /get_target_pose
    target_pose = self.navpose_dict.get(point_name)
    goal_msg = NavigateToPose.Goal()
    goal_msg.pose = target_pose
    send_goal_future = self.navclient.send_goal_async(goal_msg)

    def goal_response_callback(future):
        self.goal_handle = future.result()
        if not self.goal_handle or not self.goal_handle.accepted:
            self.get_logger().error("Goal was rejected!")
            return None

    get_result_future = self.goal_handle.get_result_async()

    def result_callback(future_result):
        self.result = future_result.result()
        self.navigation_finish_flag = True
        if self.result.status == 4:
            self.get_logger().info("Navigation finished!")
            self.res= True
```

```

        else:
            self.get_logger().info(f"Navigation failed with status:
{self.result.status}")
            self.res= False

            get_result_future.add_done_callback(result_callback)
            send_goal_future.add_done_callback(goal_response_callback)

        while not self.navigation_finish_flag:
            if self.interrupt_event.is_set() :
                self.navclient._cancel_goal(self.goal_handle)
                return None
            time.sleep(0.1)
        self.stop()
        return self.res

```

The **get_current_pose** function retrieves the robot's current map coordinates in the global coordinate system and stores these coordinates in a dictionary for easy retrieval later.

```

def get_current_pose(self):
    """
    获取当前在全局地图坐标系下的位置 /Get the current position in the global map
    coordinate system
    """
    transform = self.tf_buffer.lookup_transform("map", "base_footprint",
rclpy.time.Time())
    pose = PoseStamped()
    pose.header.frame_id = "map"
    pose.pose.position.x = transform.transform.translation.x
    pose.pose.position.y = transform.transform.translation.y
    pose.pose.position.z = 0.0
    pose.pose.orientation = transform.transform.rotation
    self.navpose_dict["zero"] = pose
    position = pose.pose.position
    orientation = pose.pose.orientation
    self.get_logger().info(f"Recorded Pose: Position: x={position.x}, y=
{position.y},z={position.z},Orientation: x={orientation.x}, y={orientation.y}, z=
{orientation.z}, w={orientation.w}")
    return True

```

